

AI IN ACTION · NGÀY 5

# Thiết kế sản phẩm AI cho sự không chắc chắn

---

Từ khả năng của model đến sản phẩm người dùng tin tưởng

VinUniversity · Day 5 · 2026

# Điểm danh - Học viên Chương trình Đào tạo Nhân tài AI thực chiến



# Phiếu khảo sát Chương trình Đào tạo AI Thực chiến



# Mục tiêu hôm nay

---

3 câu hỏi xuyên suốt ngày:

- 1. Một agent chạy được đã đủ để trở thành product chưa?**
- 2. Thiết kế product AI khác gì phần mềm truyền thống?**
- 3. Cải thiện liên tục nhờ feedback và data**

# Flow buổi học — Day 5 (08/04)

| Giờ   | Hoạt động                                             | Nội                   |
|-------|-------------------------------------------------------|-----------------------|
| 9:00  | Mở đầu — agent chạy được ≠ product thành công         |                       |
| 9:15  | AI không hoàn hảo — tự động hoá hay hỗ trợ con người? |                       |
| 9:45  | 3 trụ thiết kế + bài tập phân tích UX (cá nhân)       | Sketch giấy + 4 paths |
| 11:25 | AI Product Canvas + thực hành nhóm + review chéo      |                       |
| 14:00 | Feedback loop + data flywheel — cải thiện AI liên tục |                       |
| 14:30 | ROI 3 kịch bản: có nên đầu tư AI? + hỏi đáp           |                       |
| 16:00 | Hackathon: nhận đề, chia nhóm, bắt đầu draft SPEC     |                       |
| 23h59 | Deadline 08/04                                        | Topic + SPEC draft    |

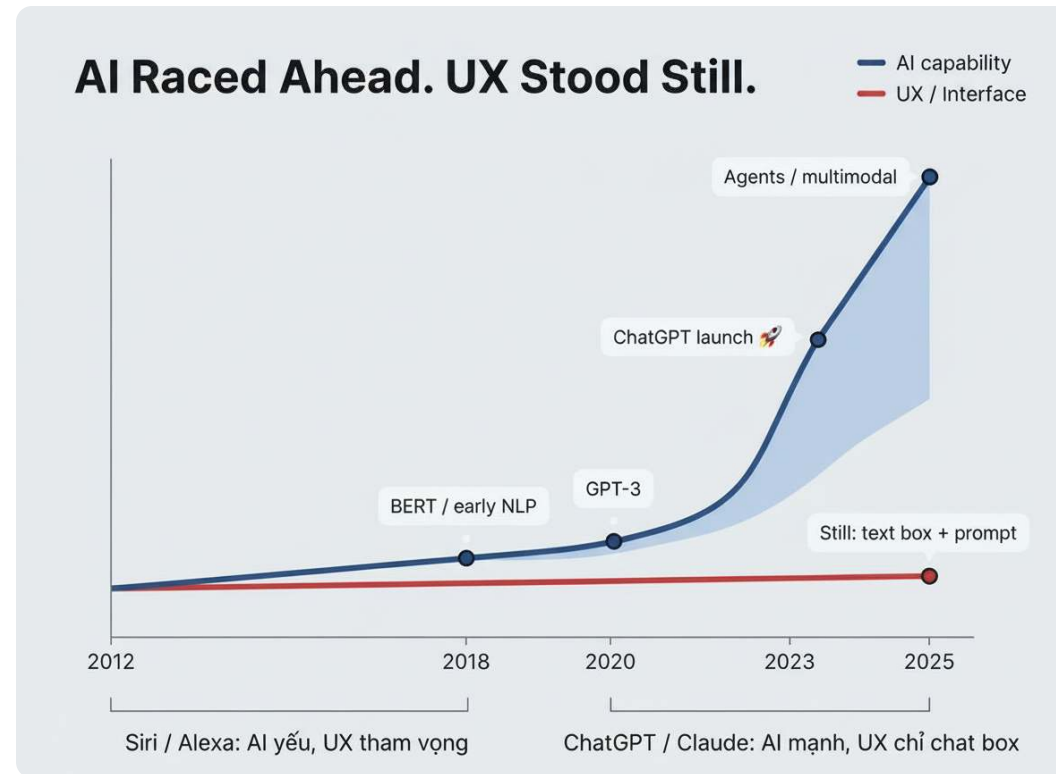
# Agent chạy được rồi — vậy tại sao có thể không ai dùng?

| # | Sai lầm                                                                    | Thực tế                                                                            |
|---|----------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| 1 | <b>Gắn AI cho có</b><br>Thêm AI vì FOMO, không vì user cần                 | Google rắc AI lên search → user bỏ qua. Perplexity cùng tech, thiết kế lại → thắng |
| 2 | <b>Chỉ build 80%</b><br>Demo ấn tượng nhưng 20% cuối mới quyết định        | Gamma/Tome tạo slide nhanh nhưng phải sửa quá nhiều → chậm hơn tự làm PowerPoint   |
| 3 | <b>Nhằm khó = giá trị</b><br>Không cần research mới, cần ứng dụng sáng tạo | Descript dùng AI có sẵn, edit video bằng cách edit text → tạo category mới         |
| 4 | <b>Tự xây AI từ đầu</b><br>Build infra riêng khi platform đủ tốt           | Builder.io chỉ train riêng 1 model cho 1 việc, còn lại dùng OpenAI                 |

## Pattern

Cùng công nghệ — khác cách build product → kết quả ngược nhau.

# Vấn đề không phải AI yếu. Vấn đề là ta đang đối xử AI như phần mềm thường.



*"AI có trí tuệ đáng kinh ngạc, nhưng giao diện vẫn chỉ như chatbot modem quay số AOL." — Aparna Chennapragada, CPO Microsoft*

## Activity nhanh: 5 câu hỏi

Nhĩ về AI agent của bạn:

1. Ai dùng?
2. Khi đúng → user được gì?
3. Khi sai → chuyện gì?
4. Bao nhiêu % đúng là đủ?
5. Mỗi lần chạy tốn gì?

Nhóm · 5 phút · Gửi lên Discord

Nội dung: brief về AI agent của bạn + trả lời 5 câu hỏi



# **AI = không chắc chắn + cách bạn handle nó**

---

Phần mềm cũ chạy theo luật. AI chạy theo xác suất.

01

# Phần mềm truyền thống vs AI product

|          | Truyền thống           | AI                                           |
|----------|------------------------|----------------------------------------------|
| Kết quả  | Luôn giống nhau        | Mỗi lần một khác                             |
| Ví dụ    | "Số dư?" → trả đúng số | "Tóm tắt email này" → mỗi lần 1 bản khác     |
| Kiểm thử | Đạt / không đạt        | Chạy 100 lần → bao nhiêu lần "đủ tốt"?       |
| Lỗi      | Bug — tìm và sửa       | Sai xác suất — không sửa được, chỉ giảm được |

*"Bạn không biết user sẽ tương tác thế nào, và cũng không biết LLM sẽ phản hồi ra sao. Cả ba — input, output, process — đều không chắc chắn."*

— [Ash Reganti, Head of AI Spotify](#) ([Lenny's Podcast](#)).

## Takeaway

Nếu sản phẩm dùng AI, bạn đang thiết kế cho **uncertainty**.

# Tại sao Gmail cần AI — và cái giá phải trả

Ví dụ kinh điển: "AI giỏi hơn nhưng không hoàn hảo"



## ⚠ Cái giá của ML:

**Chặn nhầm** → email quan trọng bị xóa, user không hề biết

**Bỏ lọt** → spam vào inbox, user thấy và tự xóa được

Không thể triệt tiêu cả hai. Chỉ chọn cái nào ít tệ hơn.

→ AI sẽ sai. Vậy 3 thứ phải thay đổi theo...

# AI sẽ sai → 3 thứ phải thiết kế khác

## 1. Requirement

**Trước:** "Click X → Y"

**Giờ:** "Hỏi X → Y ~85%, dưới 60% → hỏi lại user"

(spec ngưỡng + failure behavior)

↳ Sai thế nào là chấp nhận được?

## 2. UX

**Trước:** Thiết kế cho lúc đúng

**Giờ:** Thiết kế cho lúc **SAI**:

user thấy sai?

sửa được?

tin lại được?

(graceful failure + trust recovery)

↳ Sai thì user làm gì?

## 3. Eval

**Trước:** Đạt / không đạt

**Giờ:** Chạy 100 lần → bao nhiêu % "đủ tốt"?

PM quyết định, không phải QA

(quality distribution)

↳ Bao nhiêu % sai là chấp nhận được?

# Cách bạn deploy AI thay đổi mọi thứ

Automation — AI làm thay, user không thấy. Augmentation — AI gợi ý, user quyết định.

## Augmentation — GitHub Copilot

**30% accuracy**

→ 20M users

- Ghost text — reject = 0 friction
- Suggest-only, không auto-action
- User luôn có quyền kiểm soát

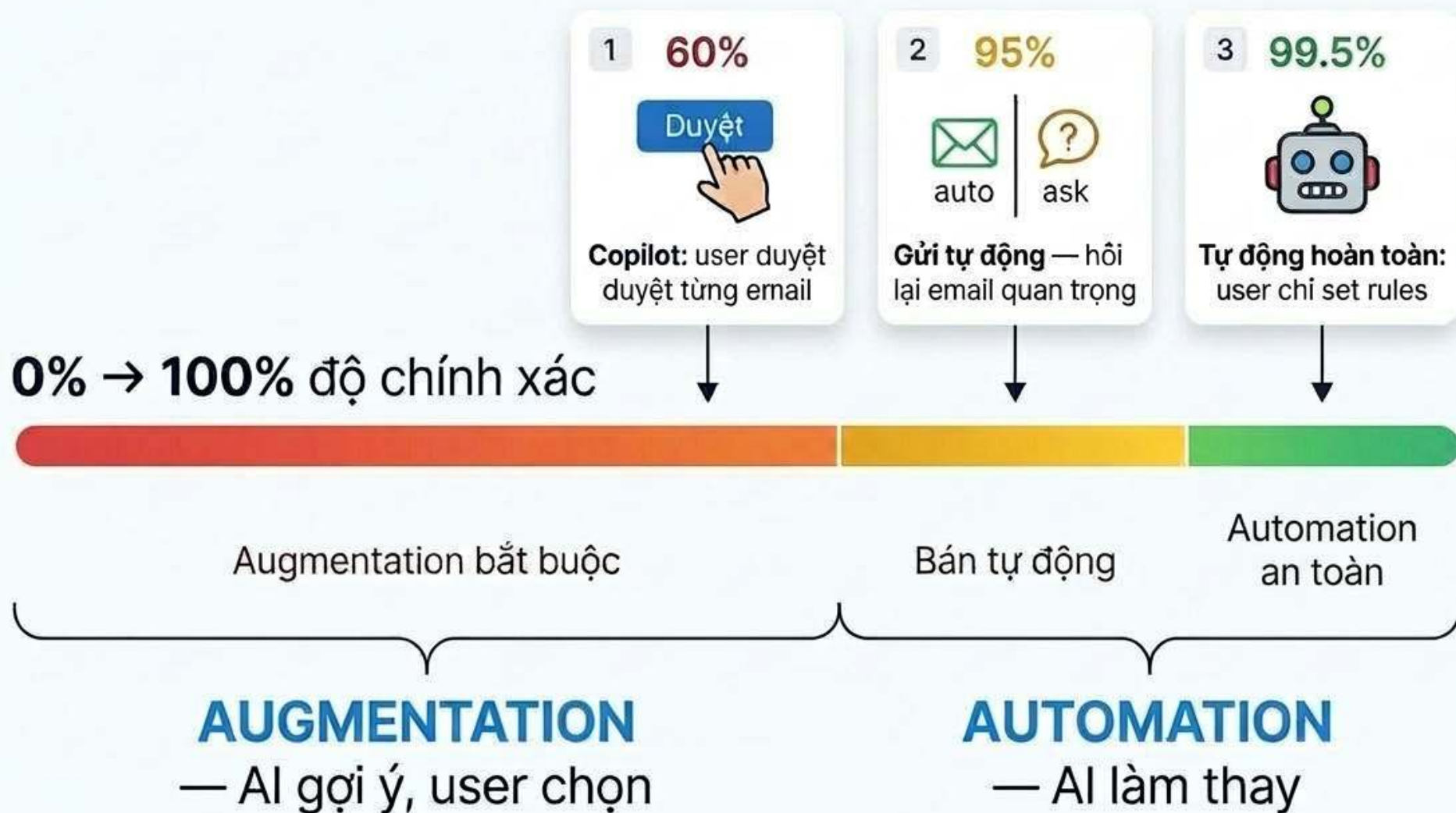
## Automation — Spam filter

User không thấy email bị xóa nhầm

- Precision cực cao bắt buộc
- False positive = mất email quan trọng
- User không có cơ hội sửa

Tại sao cùng AI mà khác nhau đến vậy? →

# Chuyển **AI Automation** và **Augmentation**



“

“Nếu model đúng 60%, bạn phải build product theo cách hoàn toàn khác.”

Kevin Weil, CEO OpenAI

# Trong thực tế: bắt đầu aug, tăng dần auto

Ví dụ: agent hỗ trợ khách hàng

| V1: Routing                                                                                                                                                                                                                                                                                                                                    | V2: Copilot                                                                                                                                                                                                                                                                                                                                                       | V3: Tự động                                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p> <b>AI làm:</b></p> <p>Phân loại + chuyển ticket đúng chỗ</p> <p> <b>Người làm:</b></p> <p>Xử lý 100% ticket</p> <hr/> <p><b>Thu:</b> data phân loại, cấu trúc prompt</p> | <p> <b>AI làm:</b></p> <p>Gợi ý draft trả lời</p> <p> <b>Người làm:</b></p> <p>Review + sửa draft</p> <hr/> <p><b>Thu:</b> % draft dùng nguyên? Phần nào bị sửa? = <b>eval miễn phí</b></p> | <p> <b>AI làm:</b></p> <p>Tự giải quyết ticket</p> <p> <b>Người làm:</b></p> <p>Chỉ xem edge cases</p> <hr/> <p><b>Thu:</b> vòng lặp tự động hoàn toàn</p> |

Mỗi version thu data để quyết định lên version tiếp → không đoán mò

 **Nhảy thẳng V3:** “Chúng tôi phải đóng product vì quá nhiều lỗi, không đếm hết các vấn đề mới phát sinh.”

— [Ash Reganti & Vikram Badam \(Lenny's Podcast\)](#).

# 3 trụ thiết kế AI product

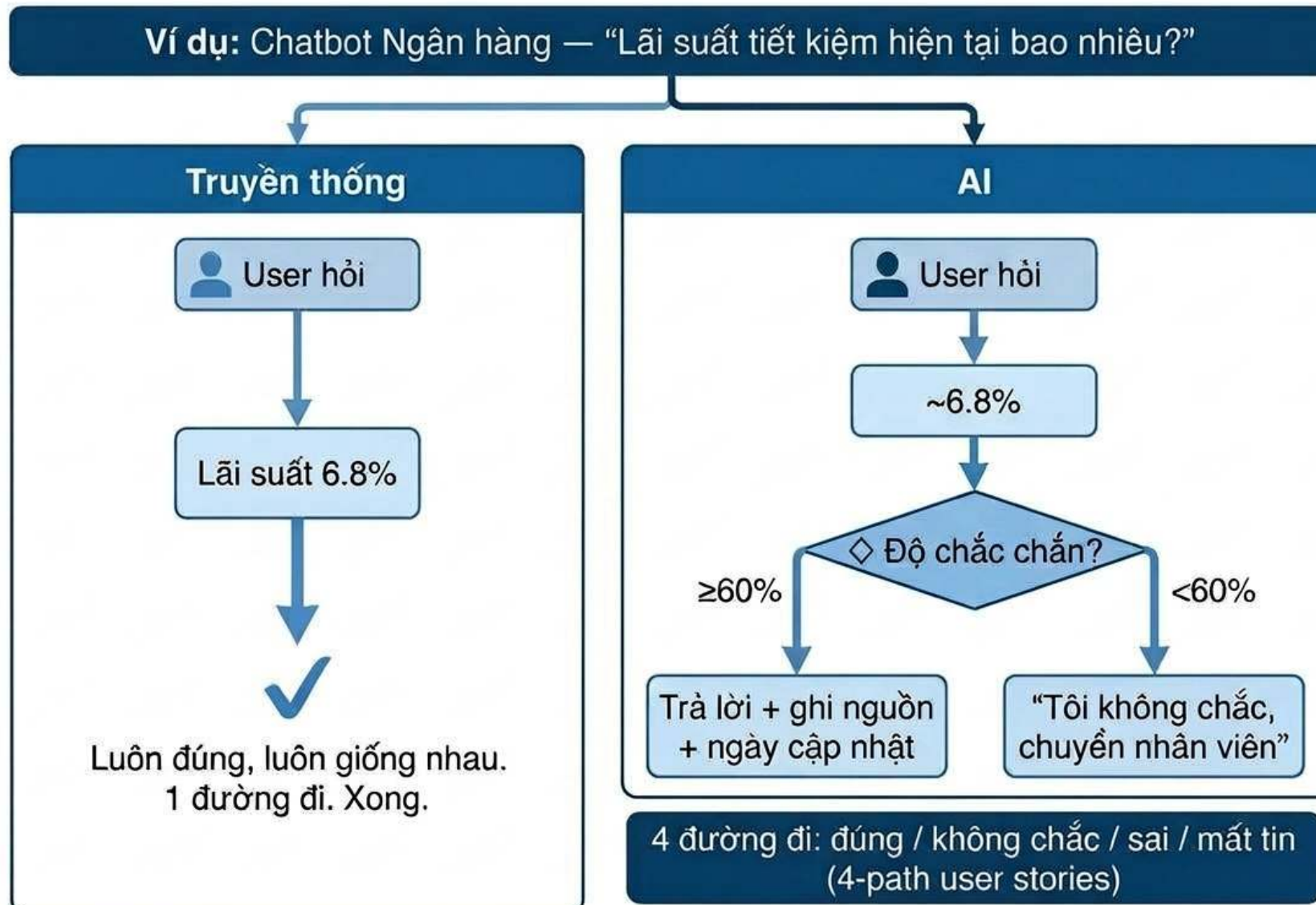
---

Yêu cầu (Requirement) · Trải nghiệm (UX) · Đánh giá (Eval)

02



## Yêu cầu (Requirement): không mô tả hành vi, mô tả kết quả + lúc sai thì sao



# Các dạng lỗi thường gặp (failure mode library)

Thay vì liệt kê tính năng → liệt kê cách sản phẩm có thể sai

**Air Canada (2024):** chatbot tự bịa chính sách hoàn vé → khách đòi đúng lời bot → tòa phán: “Bot là đại diện công ty. Phải bồi thường.”  
→ AI sai ≠ bug nhỏ. AI sai = công ty chịu trách nhiệm.

Vậy phải chuẩn bị trước — liệt kê cách AI có thể sai:

| Khi nào xảy ra<br>(trigger)                 | Chuyện gì xảy ra<br>(hậu quả)                                           | Xử lý thế nào<br>(mitigation)                                     |
|---------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------|
| Hỏi ngoài phạm vi: “Hoàn vé được không?”    | Bot bịa thông tin → khách làm theo → thiệt hại (Air Canada)             | Nhận ra → “Tôi không có thông tin này, để tôi chuyển nhân viên”   |
| Hỏi mơ hồ: “Tôi bị mất thẻ”                 | Trả lời sai ngữ cảnh — thẻ tín dụng hay thẻ thành viên? → hướng dẫn sai | Hỏi lại: “Bạn muốn hỏi về thẻ tín dụng hay thẻ thành viên?”       |
| Dữ liệu cũ: lãi suất đã thay đổi tuần trước | Trả lãi suất 7.5% (hết hạn) → khách ra quầy → chỉ còn 5.5% → mất tin    | Gắn thời gian: “Cập nhật lần cuối: 01/04. Vui lòng kiểm tra lại.” |

## Câu hỏi then chốt

Top 3 cách sản phẩm của bạn có thể sai là gì?

# Tìm lỗi trong spec

Chọn 1 đề. Bạn là product builder nhận spec này — thiếu gì để build được?

## ĐỀ 1

### Chatbot CSKH ngân hàng

“Chatbot trả lời câu hỏi khách hàng về tài khoản, thè, vay. Hiểu tiếng Việt có dấu và không dấu. Phản hồi trong 3 giây. Hỗ trợ 24/7.”

Gợi ý: user hỏi ngoài phạm vi thì sao? Bot sai thông tin lỗi suất?

## ĐỀ 2

### AI tóm tắt meeting

“AI tự động tóm tắt cuộc họp thành bullet points. Bắt action items, người chịu trách nhiệm, và gửi email tự động.”

Gợi ý: gán sai người chịu trách nhiệm? Tóm sai ý chính?

## ĐỀ 3

### AI chấm bài luận

“AI chấm điểm bài luận tiếng Việt của học sinh. Cho điểm 1–10 kèm nhận xét ngắn. Chấm xong trong 5 phút.”

Gợi ý: chấm sai vì văn phong lạ? Học sinh khiếu nại thì sao?

## ĐỀ 4

### AI lọc CV ứng viên

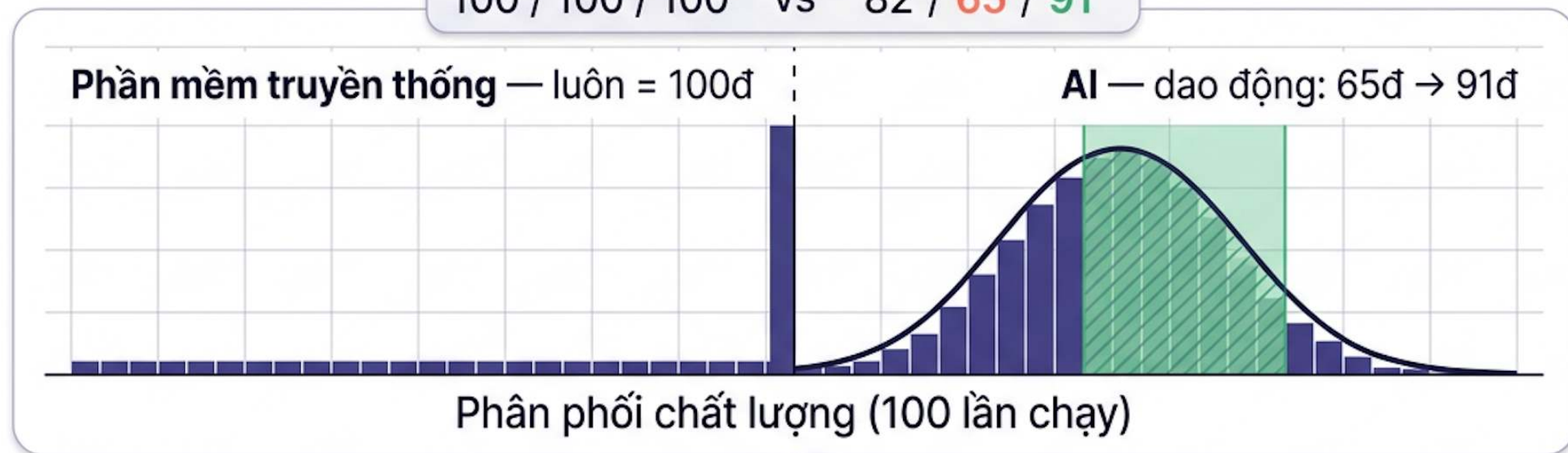
“AI đọc CV ứng viên và xếp hạng phù hợp cho vị trí. Lọc top 20 từ 500 hồ sơ. Xử lý trong 1 giờ.”

Gợi ý: loại nhầm người giỏi? Bias giới tính / trường?

Gõ vào Discord: liệt kê 1–2 cách AI có thể sai + user story path thiếu

# Đánh giá (Eval): không phải đạt/trượt, mà sai bao nhiêu là chấp nhận được?

| Đạt giá         | AI giá       |
|-----------------|--------------|
| 100 / 100 / 100 | 82 / 65 / 91 |



① Nhìn 50–100  
kết quả



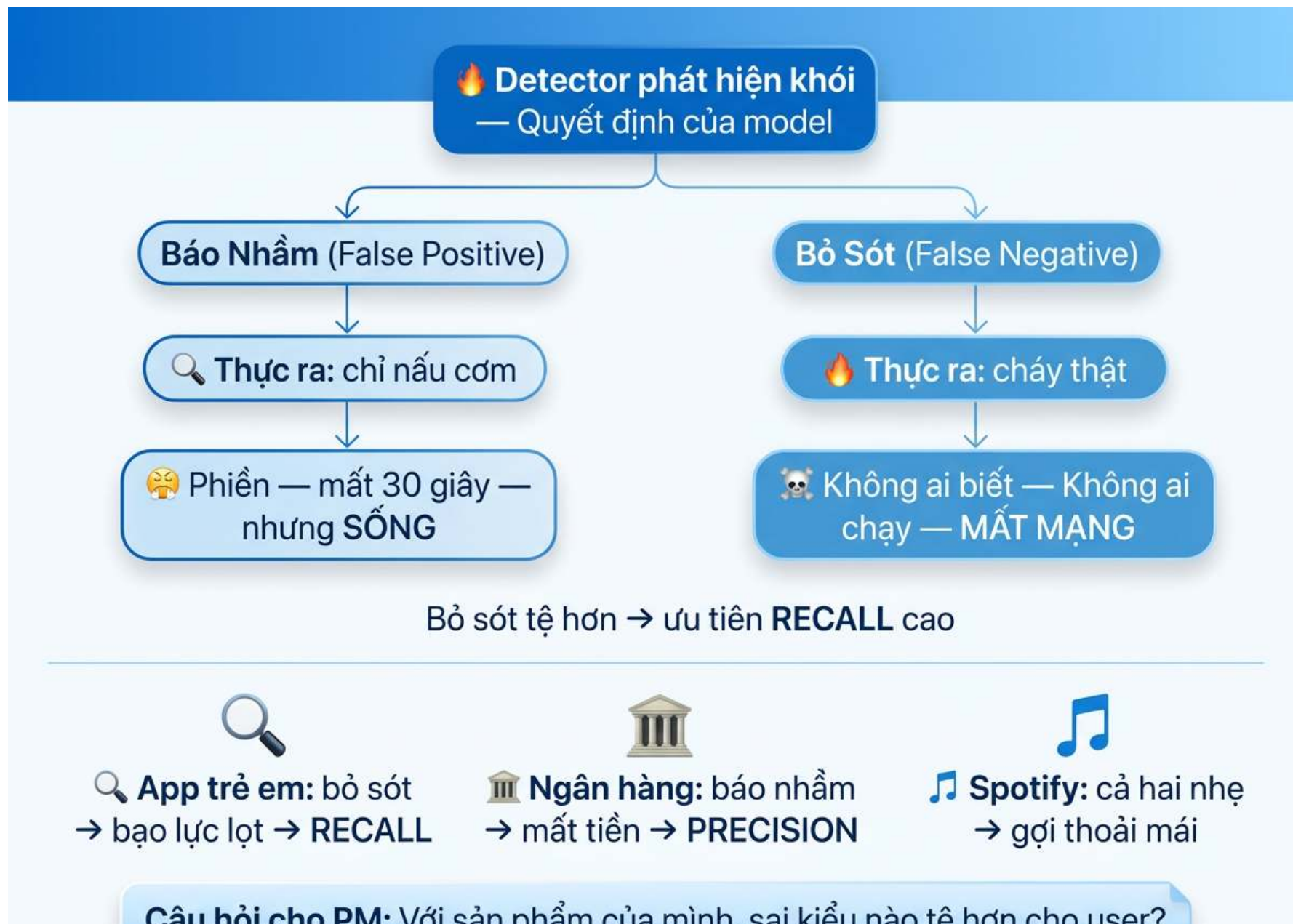
② Phân loại lỗi



③ Viết eval từ các  
dạng lỗi lặp lại

# Sai kiểu nào tệ hơn?

Precision (ít báo nhầm) hay Recall (ít bỏ sót)



# Precision vs Recall — tính thế nào?

Ví dụ: AI lọc video cho app trẻ em — 100 videos, trong đó 10 cái xấu thật sự

|           | AI đánh dấu "xấu"                     | AI cho qua                            |
|-----------|---------------------------------------|---------------------------------------|
| Xấu thật  | 8 ✓                                   | 2 × bỏ sót<br>(trẻ thấy nội dung xấu) |
| Lành thật | 5 × báo nhầm<br>(video tốt bị gỡ oan) | 85 ✓                                  |

**Precision = 8/13 = 62%**

AI đánh dấu 13 cái "xấu" — chỉ 8 cái xấu thật

→ "Khi AI nói CÓ, đúng bao nhiêu?"

**Recall = 8/10 = 80%**

10 cái xấu thật sự, AI tìm được 8 — lọt 2 cái

→ "Trong số cần tìm, AI tìm được bao nhiêu?"

2 cái lọt = 2 video xấu trẻ nhìn thấy. 5 cái báo nhầm = 5 video tốt bị gỡ oan.

Cái nào tệ hơn? → Lọt tệ hơn → cần **recall cao**.

Nhưng đổi sang duyệt vay ngân hàng: báo nhầm = duyệt sai → mất tiền → **precision cao**.

→ Không có đáp án chung. Tùy "sai kiểu nào tệ hơn cho user của mình?"



# Precision hay Recall? — phụ thuộc context

|                                       | User THẤY & sửa được                                                            | User KHÔNG thấy                                                                           |
|---------------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Sai mà act = hậu quả nặng (FP tệ hơn) | <b>PRECISION</b><br>VD: Legal RAG, Bank chatbot<br>User thấy những hậu quả nặng | <b>PRECISION</b><br>VD: Spam filter, Auto-send email<br>Sai mà không ai biết = nguy hiểm  |
| Bỏ lọt = mất giá trị (FN tệ hơn)      | <b>RECALL</b><br>VD: Copilot, FAQ chatbot<br>Gợi ý nhiều, user tự filter        | <b>RECALL</b><br>VD: Content mod (trẻ em), Fraud<br>Bỏ lọt = thảm họa, dù user không thấy |

## Rule of thumb

Sai mà user KHÔNG BIẾT → thường precision. Nhưng nếu bỏ lọt = thảm họa → recall bất kể user thấy hay không.

# Precision hay recall?

Mỗi sản phẩm nên ưu tiên **precision** (đừng nhầm) hay **recall** (đừng sót)?

- 1 AI lọc nội dung cho app trẻ em
- 2 Code autocomplete (kiểu Copilot)
- 3 AI đọc X-quang phát hiện ung thư
- 4 AI duyệt khoản vay ngân hàng
- 5 AI gợi ý bài hát (Spotify)

**Gõ vào Discord: số-P hoặc R — lý do 1 câu**

Ví dụ: 1-R — vì bỏ sót = trẻ thấy nội dung xấu



# 4 câu mọi AI product phải trả lời

| Câu hỏi |                                   | Ví dụ: Copilot                                             |
|---------|-----------------------------------|------------------------------------------------------------|
| 1       | Khi đúng → user thấy gì?          | Gợi code màu xám → bấm Tab để chấp nhận                    |
| 2       | Khi không chắc → hệ thống làm gì? | Gợi ngắn hơn, ít tự tin hơn → user tự viết tiếp            |
| 3       | Khi sai → user sửa thế nào?       | Tiếp tục gõ = gợi ý biến mất. Cost = 0.                    |
| 4       | Khi mất tin → gỡ thế nào?         | Tắt Copilot cho file này / ngôn ngữ này. Bật lại khi muốn. |

## Tại sao Copilot 30% accuracy mà 20M người dùng?

Vì cả 4 câu đều có câu trả lời tốt. Sai → không mất gì. Đúng → tiết kiệm thời gian.

**Microsoft Tay (2016):** không trả lời được câu nào → bị troll thành racist bot trong vài giờ.

# Khi AI sai: xử lý nhẹ nhàng (graceful failure) + giữ niềm tin (trust recovery)

## Xử lý lỗi nhẹ nhàng

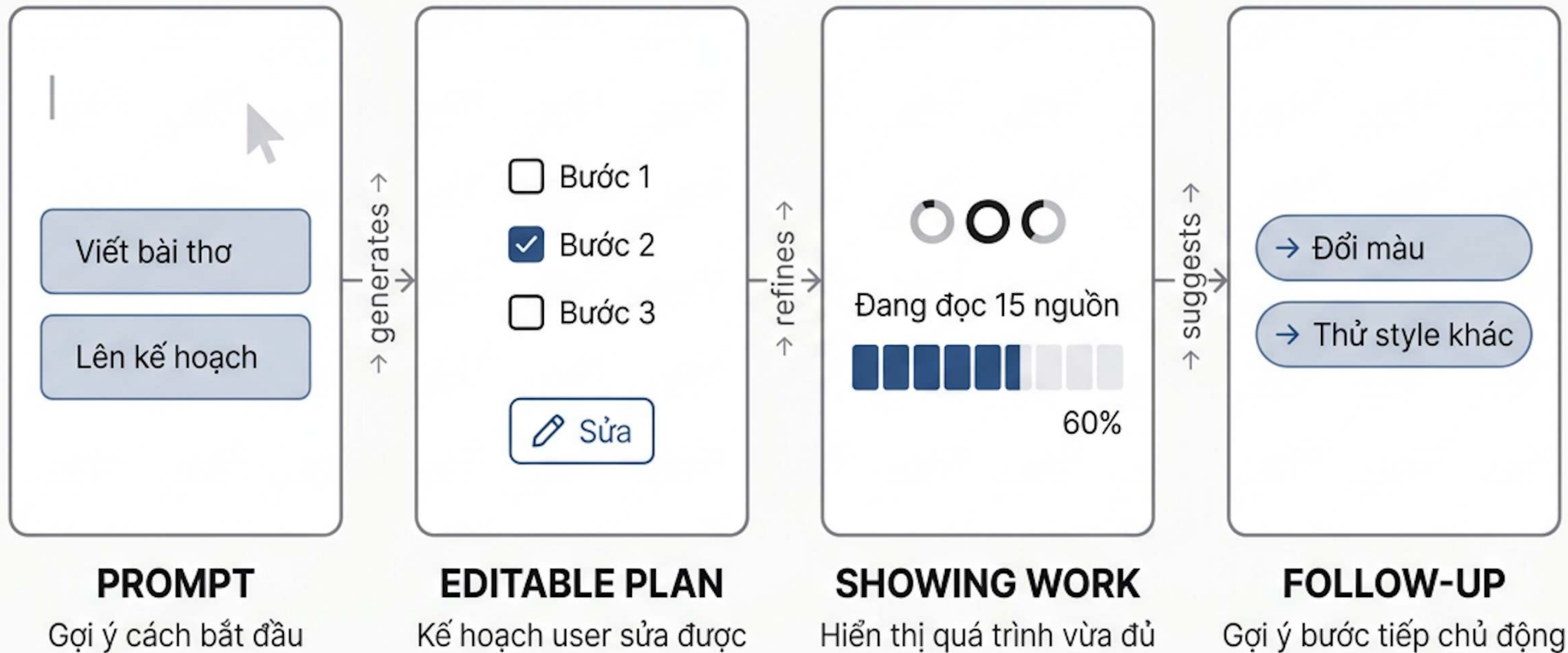
- ① Đưa nhiều lựa chọn, không chỉ 1 đáp án
- ② Cho user sửa được
- ③ Ghi nhận đã sửa  
→ AI học từ đó

## Giữ niềm tin

- ① Giải thích tại sao:  
Grammarly: "Wordiness — thử 'thrilled' thay 'very happy'"  
→ user hiểu lý do → tin và học theo
- ② Hiện độ chắc chắn:  
Kayak: "Advice: MUA — Confidence: 79%"  
→ user tự quyết
- ③ Cho lựa chọn khác

# 4 thành phần giao diện mới cho AI

Aparna Chennapragada, CPO Microsoft



"Quá dài dòng = cron job. Quá ngắn gọn = không biết AI đi đúng hướng không."

— Aparna Chennapragada, CPO Microsoft

# Bài tập UX: phân tích sản phẩm AI thật

Chọn 1 sản phẩm → dùng thử → phân tích 4 paths → sketch cải thiện → chia sẻ + vote

CHỌN 1 SẢN PHẨM

**MoMo — Trợ thủ AI Moni**

Phân loại chi tiêu tự động, chatbot tài chính cá nhân. App MoMo.

**VN Airlines — Chatbot NEO**

Chatbot hỗ trợ KH, tra cứu chuyến bay. vietnamairlines.com hoặc Zalo VNA.

**V-App — Trợ lý ảo V-AI**

Trợ lý voice/text, gợi ý theo context. App V-App.

15'

**Khám phá**

Tìm hiểu marketing  
rồi dùng thử app  
Marketing khớp thực tế?

10'

**Phân tích 4 paths**

Đúng → user thấy gì?  
Không chắc → hỏi lại?  
Sai → sửa thế nào?  
Mất tin → có exit?

10'

**Sketch to-be**

Chọn 1 path yếu nhất  
Vẽ as-is (trái) + to-be (phải)  
Thêm gì? Bớt gì? Đổi gì?

5'

**Share + vote**

Chia sẻ trong nhóm  
Mỗi người 30 giây  
Vote sketch hay nhất → bonus

**Nộp cá nhân:** sketch giấy (as-is + to-be) + ghi chú 4 paths. Điểm cá nhân 10đ. Vote = bonus 2đ.

Nice to have: screenshot app + annotate chỗ hay / chỗ gãy.



Đề bài chi tiết: [github.com/VinUni-AI20k/Day05-AI-Product-Labs](https://github.com/VinUni-AI20k/Day05-AI-Product-Labs)

# Debrief: kết nối 3 trụ

---

**Requirement:** spec thiếu gì? (Discord A — 4 specs đều thiếu cùng pattern)

**Eval:** precision hay recall? (Discord C — phụ thuộc context, không phải kỹ thuật)

**UX:** path nào yếu nhất? (Workshop — sản phẩm thật đều có gap)

**3 trụ không độc lập:**

**Auto/Aug → Requirement khác → UX khác → Eval khác**

Đây là vòng tròn, không phải đường thẳng.

**→ Tiếp theo: Canvas** — gom requirement + UX + eval thành 1 trang product spec.

**Nghỉ giải lao — 15 phút**

# Canvas: gom lại thành lightweight spec

---

Gộp 3 trụ thành 1 trang product spec

03

# AI Product Canvas — 3 cột

| VALUE                                                                                                                      | TRUST                                                                                                                                        | FEASIBILITY                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>■ User nào? Pain nào?</li><li>■ Auto hay Aug?</li><li>■ Value khi AI đúng?</li></ul> | <ul style="list-style-type: none"><li>■ Precision hay recall?</li><li>■ Khi sai → user biết/sửa thế nào?</li><li>■ Trust recovery?</li></ul> | <ul style="list-style-type: none"><li>■ Cost / latency?</li><li>■ Dependency / risk chính?</li></ul> |

**Learning Signal** (Block 4): (1) User sửa AI → data đi vào đâu? (2) Đang tốt lên hay tệ đi? (3) Càng nhiều data càng tốt?

Ví dụ: Chatbot CSKH ngân hàng

|                                                                                                                                                                           |                                                                                                                                                          |                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VALUE</b><br><b>User:</b> KH hỏi tài khoản, thẻ, vay<br><b>Pain:</b> chờ tổng đài 10–15 phút<br><b>Aug:</b> bot gợi ý, NV xử lý phức tạp<br><b>Value:</b> trả lời 24/7 | <b>TRUST</b><br><b>Precision cao:</b> sai = mất tiền (Air Canada)<br><b>Khi sai:</b> “Để tôi chuyển NV”<br><b>Recovery:</b> “Xin lỗi, NV sẽ liên hệ lại” | <b>FEASIBILITY</b><br><b>Cost:</b> ~\$0.02/lượt GPT-4o<br><b>Latency:</b> <3 giây<br><b>Risk:</b> lỗi suất cũ → sai thông tin<br><b>Dep:</b> API ngân hàng real-time |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Learning Signal:** ① Log mỗi lần chuyển NV + lý do ② % câu hỏi bot tự giải quyết (tuần/tuần) ③ Mỗi lần chuyển NV = 1 ví dụ train thêm



## **Nhóm sketch Canvas outline**

Sketch Canvas cho project của nhóm

Chưa cần đầy đủ — đánh dấu [?] cho chưa biết

**Sketch (15 min) + Peer peek (10 min)**

Nhóm · 25 phút

# Feedback Loop + Data Flywheel

---

Sau khi launch, sản phẩm phải học hoặc sẽ xuống cấp.

04

# Feedback loop: truyền thống vs AI

|               | Truyền thống (vật thể)            | AI (sinh vật)                           |
|---------------|-----------------------------------|-----------------------------------------|
| Bản chất      | Vật thể tĩnh — xuất xong = xong   | Sinh vật sống — xuất xong = mới bắt đầu |
| Phản hồi      | Thủ công, theo sprint/quý         | Liên tục, tự động                       |
| Cải thiện     | Team phân tích data → cập nhật    | Model tự học từ tương tác               |
| Tài sản riêng | Tính năng, thương hiệu, mạng lưới | Vòng data + model tinh chỉnh            |



**Hiệu ứng Alexa:** User hỏi 3 lần, AI sai cả 3 → ngừng dùng vĩnh viễn. Không phản hồi = không cải thiện.

# 3 loại feedback signal

Sản phẩm AI cần thu signal từ user để cải thiện. 3 cách thu:

## Ngầm (implicit)

User không cố ý phản hồi — hệ thống tự thu:

- Thời gian xem, cuộn trang, bỏ qua
- Chấp nhận / từ chối gợi ý

VD: Copilot đo % Tab (chấp nhận) vs Esc (bỏ qua)

VD: YouTube đo thời gian xem > click

## Trực tiếp (explicit)

User chủ động đánh giá:

- Thích / không thích (thumbs up/down)
- Đánh giá sao, báo cáo, đánh dấu

VD: ChatGPT — nút 👍👎 mỗi câu trả lời

VD: MoMo — “Phân loại này đúng không?”

## Sửa lỗi (correction)

User trực tiếp sửa kết quả AI:

- Sửa text AI viết, đánh dấu sai
- Ghi đè quyết định của AI

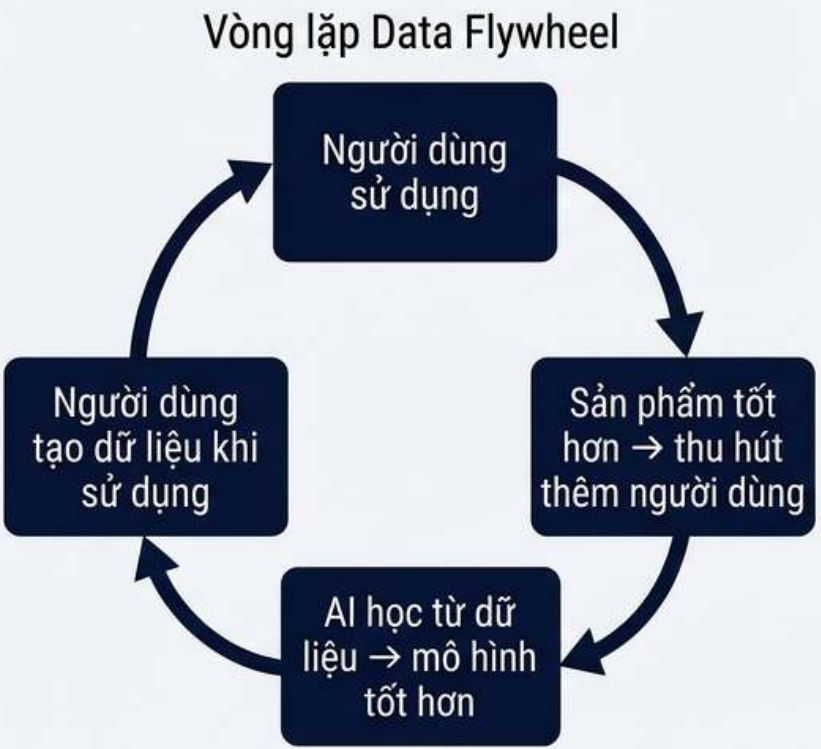
VD: Grammarly — user bỏ gợi ý = “sai”

VD: Gmail Smart Reply — user sửa rồi gửi

**Câu hỏi cho SPEC:** sản phẩm của bạn thu signal loại nào? Nếu chưa có → thiết kế ngay từ đầu.

# Data Flywheel

## Data Flywheel



## Sai lầm phá vỡ Flywheel

- ❌ **Không thiết kế thu dữ liệu từ đầu**  
không có nút "sai/đúng" cho user bấm
- ❌ **Thu sai thứ**  
chỉ đếm lượt dùng, không thu user sửa AI chỗ nào
- ❌ **Thu rồi nhưng không đưa về model**  
log đầy nhưng không ai dùng để cải thiện

**Takeaway:** AI ai cũng dùng được (commodity). Chỉ dữ liệu riêng + cách dùng riêng = lợi thế thật.

## MidJourney



## Microsoft Dragon

|                      | Phiên bản V1              | Phiên bản V2                              | Phiên bản V3                               |
|----------------------|---------------------------|-------------------------------------------|--------------------------------------------|
| <b>Nguồn dữ liệu</b> | Dữ liệu tổng hợp          | 600K ca thực tế + chuyên gia đánh giá     | Liên tục tối ưu                            |
| <b>Độ chính xác</b>  | ■■■■■■■■ 30-60%           | ■■■■■■■■■■ ~75%                           | ■■■■■■■■■■■ 83% ✓                          |
| <b>Kết quả</b>       | Dữ liệu giả -> kết quả tệ | Dữ liệu thật + người đánh giá -> nhảy vọt | Bác sĩ dùng hàng ngày -> data mới mỗi ngày |

Loop liên tục: dữ liệu mới -> fine-tune -> tốt hơn nữa -> thêm dữ liệu

# Con số đẹp chưa đủ — phải biết sai ở đâu

Định tính trước, định lượng sau

AI phân loại email: 87% đúng — nghe ổn? Nhưng 13% sai ở đâu?

Email FYI phân loại nhầm

→ Phiền, không sao 🙄

Email từ khách VIP phân loại nhầm vào “spam”

→ **Mất deal 500 triệu** 💀

Cùng 13% sai — nhưng hậu quả khác nhau hoàn toàn. → Phải nhìn LỖI CỤ THỂ trước khi nhìn CON SỐ.

4 cách cải tiến (dùng trong hackathon):

1. Sửa prompt

Viết rõ hơn, bớt mập mờ

VD: “Phân loại email thành 3 loại: urgent / action / FYI” thay vì “phân loại email”

2. Sửa dữ liệu

Thêm data đúng, bớt data nhiễu

VD: thêm 50 email mẫu từ khách VIP để AI nhận diện pattern

3. Sửa trải nghiệm (UX)

Cho user thấy AI không chắc + sửa được

VD: email AI không chắc → highlight vàng: “Kiểm tra lại email này?”

4. Sửa cách đo (eval)

Đo đúng thứ quan trọng

VD: không đo “87% đúng” mà đo “bao nhiêu email urgent bị phân loại nhầm?”

## Hackathon

Nhìn kết quả AI trước (định tính) → thấy lỗi gì → rồi mới đo bao nhiêu % (định lượng).  
Ít nhất 1 vòng: chạy AI → nhìn output → tìm lỗi → sửa bằng 1 trong 4 cách trên → chạy lại.

# 8 loại dữ liệu giá trị cao

## 1. Dữ liệu thời gian thực

Giá cổ phiếu, giao thông, cảm biến IoT

VD: Grab dùng traffic data để dự đoán giá cước

## 2. Dữ liệu riêng user

Lịch sử mua, sở thích, hành vi dùng app

VD: Netflix gợi ý phim theo lịch sử xem

## 3. Dữ liệu chuyên ngành

Y tế, pháp lý, tài chính — không có sẵn trên internet

VD: Dragon dùng 600K ca bệnh thật để train

## 4. Dữ liệu đánh giá của người

Nhãn, xếp hạng, chú thích từ chuyên gia / user

VD: bác sĩ đánh giá kết quả AI đọc X-quang

## 5. Học tăng cường (RLHF)

User chọn A vs B → model học “cái nào tốt hơn”

VD: ChatGPT thu thumbs up/down để fine-tune

## 6. Dữ liệu ngữ cảnh

Vị trí, thời gian, thiết bị, tình huống dùng

VD: Google Maps gợi ý khác nhau lúc 8h sáng vs 8h tối

## 7. Dữ liệu tích hợp

Kết nối từ CRM, ERP, API nội bộ

VD: Salesforce Einstein dùng data CRM để dự đoán deal

## 8. Dữ liệu tổng hợp

Tạo tự động bằng AI, dùng để tăng cường data thật

VD: Dragon V1 dùng data giả → chỉ 30–60% → phải thay bằng data thật

# ROI: tính bằng range, không 1 số

---

AI ROI không cố định — phải tính 3 kịch bản.

05



# ROI: truyền thống vs AI

|         | Truyền thống                    | AI                                                                          |
|---------|---------------------------------|-----------------------------------------------------------------------------|
| ROI     | Tính 1 số                       | Tính 3 kịch bản (conservative / realistic / optimistic)                     |
| Mở rộng | Chi phí/đơn vị giảm khi mở rộng | Chi phí/đơn vị có thể tăng theo lượng dùng (suy luận, data, huấn luyện lại) |
| "Xong"  | Ship = xong                     | Ship = mới bắt đầu (cần maintain + improve)                                 |

## Chi phí

Suy luận + data + duy trì

## Năng lực

Độ chính xác + phạm vi + tốc độ

## Tốc độ

Thời gian tạo giá trị + vòng lặp

# 3 chiến lược ROI

---

## Copilot — Ưu tiên tốc độ

- Giảm thời gian code 55%
- ROI = giờ lập trình viên tiết kiệm
- Chấp nhận độ chính xác vừa đủ

## Harvey — Ưu tiên năng lực

- AI cho luật sư
- ROI = vụ mới không thể làm trước
- Precision cực cao bắt buộc

## Grammarly — Nhạy chi phí

- 30M+ free users
- ROI = chi phí mỗi lần sửa phải thấp
- Tối ưu chi phí suy luận

# ROI cho hackathon SPEC

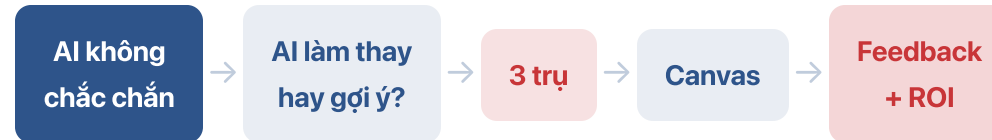
SPEC deliverable phải có ROI 3 kịch bản:

| Kịch bản   | Giả định                             | ROI           |
|------------|--------------------------------------|---------------|
| Thận trọng | Độ chính xác thấp, áp dụng chậm      | [Tính cụ thể] |
| Thực tế    | Độ chính xác trung bình, áp dụng vừa | [Tính cụ thể] |
| Lạc quan   | Độ chính xác cao, flywheel hoạt động | [Tính cụ thể] |

## Lưu ý

Không cần số chính xác. Cần logic rõ ràng và giả định hợp lý.

# Tổng kết: chuỗi domino



① **AI không chắc chắn** — kết quả mỗi lần một khác, không phải phần mềm đúng 100%

② **AI làm thay hay gợi ý?** — quyết định này ảnh hưởng mọi thứ phía sau

③ **3 trụ:**

Yêu cầu (Requirement) — mô tả kết quả + lúc sai thì sao

Trải nghiệm (UX) — thiết kế cho lúc sai

Đánh giá (Eval) — sai bao nhiêu là chấp nhận được?

④ **Canvas** — gom 3 trụ thành 1 trang để bắt đầu

**Feedback** — sản phẩm phải học, không thì xuống cấp

⑤ **ROI** — tính 3 kịch bản (thận trọng / thực tế / lạc quan), không phải 1 số

# Hackathon 2 ngày: từ SPEC đến Demo

---

Day 5 16:00–18:00 → Day 6 cả ngày

HACK

# Timeline hackathon

| Thời gian          | Hoạt động                                                        | Deliverable                       |
|--------------------|------------------------------------------------------------------|-----------------------------------|
| 08/04 — 16:00      | Briefing hackathon + chia nhóm + chọn track + bắt đầu draft SPEC |                                   |
| 08/04 — 23h59      | Deadline nộp SPEC draft                                          | Topic + SPEC draft trên repo nhóm |
| 09/04 — 09:00 (M1) | GV review SPEC draft + phản hồi từng nhóm                        |                                   |
| 09/04 — 11:00 (M2) | “Show 1 thứ chạy được” — demo nhanh 1 AI call thật               |                                   |
| 09/04 — 13:00 (M3) | SPEC final + demo flow draft — chuẩn bị bài trình bày            |                                   |
| 09/04 — 15:30 (M4) | Demo prep xong + dry run với nhóm bên cạnh                       |                                   |
| 09/04 — 16:00 (M5) | Gallery walk + demo round — trình bày tại bàn                    | Prototype deadline                |
| 09/04 — 17:00 (M6) | Top teams present + trao giải + closing                          |                                   |

# Scoring: 60% nhóm / 40% cá nhân

| Hạng mục                            | Loại                    | Điểm |
|-------------------------------------|-------------------------|------|
| SPEC milestone                      | Nhóm (20) + Cá nhân (5) | 25   |
| Prototype milestone                 | Nhóm (10) + Cá nhân (5) | 15   |
| Demo Day                            | Nhóm                    | 25   |
| Bài tập UX                          | Cá nhân + bonus         | 10   |
| Individual form (role + reflection) | Cá nhân                 | 25   |
| Tổng                                |                         | 100  |

# 5 tracks + SPEC deliverable

## Chọn 1 track

| Track | Lĩnh vực           | Gợi ý                                |
|-------|--------------------|--------------------------------------|
| A     | VinFast            | AI cho ô tô / di chuyển / hậu mãi    |
| B     | Vinmec             | AI cho y tế / trải nghiệm bệnh nhân  |
| C     | VinUni – VinSchool | AI cho giáo dục / vận hành trường    |
| D     | XanhSM             | AI cho vận tải / hành khách / tài xế |
| E     | Tự chọn            | Tự chọn, phải có user thật           |

## SPEC draft — 6 items

- AI Product Canvas (Value / Trust / Feasibility + Learning Signal)
- User Stories × 4 paths (Happy / Low-confidence / Failure / Correction)
- Eval metrics: 3 metrics + threshold + red flag
- Top 3 failure modes (Trigger / Hậu quả / Mitigation)
- ROI 3 kịch bản (conservative / realistic / optimistic)
- Mini AI spec 1 trang

Template: **hackathon-toolkit/01-templates/** trên repo nhóm



# Prototype deliverable — 3 levels

## Sketch

- Vẽ / draft flow trên giấy, slides, whiteboard
- Chưa build gì

**Đủ điểm**

## Mock prototype

- UI/flow build được (Antigravity, Claude, v0, Figma...)
- Chưa gắn AI thật

**Bonus điểm**

## Working prototype (bonus++)

- Có AI chạy thật: input → AI → output
- Demo live được

**Bonus++ điểm**

Sketch + Mock: vẫn phải có ít nhất 1 prompt/AI call chạy thật kèm theo.

# Vòng demo — quy định

---

1. Mỗi nhóm demo tại bàn mình (laptop + poster/slides)
2. 2–3 người ở lại trình bày, 2–3 người đi xem nhóm khác
3. Feedback đủ các team trong zone — dùng feedback form (3 tiêu chí × 1-5)
4. Nộp đủ feedback forms — chấm cả chất lượng review

**Vibe-coding warning:** Demo đẹp nhưng không có SPEC / logic = 0 điểm SPEC. SPEC quan trọng hơn code.



# Chia nhóm + chọn track

Tạo nhóm 3–5 người · Chọn 1 track · Bắt đầu brainstorm bài toán

| Track | Lĩnh vực           | Gợi ý                                |
|-------|--------------------|--------------------------------------|
| A     | VinFast            | AI cho ô tô / di chuyển / hậu mãi    |
| B     | Vinmec             | AI cho y tế / trải nghiệm bệnh nhân  |
| C     | VinUni – VinSchool | AI cho giáo dục / vận hành trường    |
| D     | XanhSM             | AI cho vận tải / hành khách / tài xế |
| E     | Tự chọn            | Tự chọn lĩnh vực, phải có user thật  |

16:00 → bắt đầu ngay

# Deadline: 23:59 hôm nay (08/04)

---

## Submit trước 23:59 tối nay:

- Topic chốt — problem statement 1 câu
- SPEC draft — Canvas + hướng đi chính
- Phân công nhóm ai làm gì

**Không cần hoàn chỉnh — draft đủ để GV review sáng Day 6.**